ZIPGUARD



Documentation

Comprendre les bombes de décompression
pour mieux s'en protéger.

Evan · @Picsou06 • Hamza Karrouchi · @CodeByHaamza

16/10/2026

Sommaire

1 **Comment ça marche** — le mécanisme des zip bombs

2 **Prévention** — six garde fous et la mesure

3 **Anatomie du dépôt** — le code et la frontière de sécurité

CHAPITRE 1

Comment fonctionne une bombe de décompression

> *Objectif pédagogique : comprendre le mécanisme pour mieux s'en protéger. Ce dépôt ne fournit aucune bombe prête à l'emploi.*

Le principe

Une **bombe de décompression** (ou *zip bomb*) est une archive volontairement minuscule qui, une fois décompressée, produit une quantité de données énorme, suffisante pour saturer la mémoire vive ou le disque de la machine qui l'ouvre. C'est une attaque par **déni de service** : elle ne vole rien, elle épuise une ressource.

Tout repose sur le **ratio de compression**. Les algorithmes sans perte comme DEFLATE remplacent les répétitions par de courtes références. Un fichier d'un gigaoctet composé du même octet répété se compresse en quelques kilooctets, car l'algorithme n'a qu'à encoder « cet octet, un milliard de fois ».

Les trois familles

Famille	Idée	Détection
Fichier unique très répétitif	Un seul membre énorme et ultra redondant	Ratio de compression anormal
Archive imbriquée	Des archives dans des archives, sur plusieurs niveaux	Profondeur et présence de `.zip` internes
Recouvrement d'entrées	Plusieurs entrées pointant vers les mêmes données compressées	Somme des tailles annoncées contre taille réelle

Pourquoi c'est dangereux

Un décompresseur naïf lit l'en-tête, voit un petit fichier, et commence à écrire la sortie sans jamais vérifier combien d'octets il a déjà produits. La mémoire ou le disque se remplit avant qu'il ne s'arrête. Le service tombe.

La bonne nouvelle : toutes ces familles partagent une signature mesurable **avant** d’écrire quoi que ce soit. C’est exactement ce que lit le scanner de ce dépôt. Voir 02-prevention.md.

CHAPITRE 2

Se protéger des bombes de décompression

La règle d'or : **ne jamais faire confiance aux métadonnées, toujours mesurer en lisant.**

Les six garde fous

1. **Plafond de taille décompressée totale.** Additionner les `file_size` annoncés et refuser au delà d'un seuil. ZipGuard : `max_total_bytes`.
2. **Plafond par entrée.** Un seul membre ne doit pas pouvoir dépasser une limite. ZipGuard : `max_entry_bytes`.
3. **Ratio de compression maximal.** Taille décompressée divisée par taille compressée. Un ratio de 1000x est presque toujours une attaque. ZipGuard : `max_ratio`.
4. **Nombre d'entrées maximal.** Des millions de fichiers vides épuisent aussi les ressources. ZipGuard : `max_entries`.
5. **Profondeur d'imbrication maximale.** Interdire ou limiter les archives dans les archives. ZipGuard : `max_depth`.
6. **Lecture plafonnée pendant l'extraction.** Même après validation, compter les octets écrits et couper au dépassement, car l'en-tête peut mentir. ZipGuard : `safe_extract`.

Pourquoi mesurer pendant la lecture

Les champs de taille d'un ZIP sont déclaratifs : rien n'oblige une archive malveillante à dire la vérité. Un attaquant peut annoncer une petite taille et fournir un flux qui ne s'arrête jamais. La seule défense fiable est de **compter les octets réellement produits** et de s'arrêter net au dépassement, bloc par bloc. C'est le rôle de `_stream_entry`.

Bonus : le Zip Slip

Au passage, un décompresseur doit refuser les noms d'entrée qui sortent du dossier cible (par exemple `../../etc/passwd`). ZipGuard résout chaque chemin et vérifie qu'il reste contenu dans la racine autorisée (`Scanner._resolve`).

En production

- Décompresser dans un bac à sable isolé (conteneur en lecture seule, quota disque).
- Journaliser chaque refus avec sa raison.
- Traiter les archives dans un worker séparé que l'on peut tuer sans affecter le service.

CHAPITRE 3

Anatomie du dépôt

```
zipguard/
├── README.md                 présentation animée et guide de prise en main
├── compose.dev.yaml          orchestration de développement
├── compose.prod.yaml         orchestration de production durcie
├── docs/                     cours et explications
│   ├── 01-comment-ca-marche.md
│   ├── 02-prevention.md
│   └── 03-anatomie.md
├── samples/                  archives de démonstration générées localement
└── zipguard.app/             service applicatif (code + Docker)
    ├── Dockerfile
    ├── pyproject.toml
    ├── src/zipguard/
    │   ├── limits.py         seuils de sécurité immuables
    │   ├── scanner.py        inspection et extraction plafonnée
    │   ├── demo.py           générateur de démonstration borné
    │   ├── report.py         rendu lisible d'un verdict
    │   └── __main__.py       interface en ligne de commande
    └── tests/                couverture du comportement
```

Le flux d'une inspection

1. `__main__.scan` lit les arguments et construit un objet `Limits`.
2. `Scanner.inspect` ouvre l'archive, lit les métadonnées, calcule les statistiques.
3. Chaque seuil dépassé ajoute une raison ; un `Verdict` est rendu.
4. `report.render` transforme le verdict en rapport lisible.
5. Le code de sortie vaut `0` si l'archive est sûre, `1` sinon : utilisable dans un script.

La frontière de sécurité

Le module `demo.py` plafonne la taille à 100 Mo et ne produit qu'une seule entrée, non imbriquée, qui se décompresse exactement à la taille annoncée. Il illustre le ratio de compression sans jamais pouvoir servir d'arme. Toute la logique offensive est donc absente du dépôt : il n'y a que de la mesure et de la défense.